PYTEST · PRACTICE SESSION

Practice Tasks: Testing with pytest

Nine short exercises, from your first assert to fixtures and parametrize



Lecturer, Department of Computer Science
Denning Institute of Entrepreneurship and Technology

INSTRUCTIONS

Before you start

1. Set up the folder

```
practice/
├── mymath.py          # given code (Tasks 1, 2, 5, 6)
├── text_tools.py      # Task 3
├── list_tools.py      # Task 4
├── cart.py            # Task 7
├── file_tools.py      # Task 8
├── test_task1.py      # you write these
├── test_task2.py
└── ...
```

2. Run your tests

```
pip install pytest
pytest -v                  # run everything
pytest -v test_task3.py    # run one file
```

Rules for every task

- Copy the given code into its `.py` file exactly as shown
- Put your tests in a file named `test_taskN.py`
- Every test function name must start with `test_`
- Write **at least the number of tests asked for**, more is welcome
- A task is done when `pytest -v` shows all green

Difficulty

EASY Tasks 1 to 4, plain `assert` and `pytest.raises`

MEDIUM Tasks 5 to 8, `approx`, parametrize, fixtures

CHALLENGE Task 9, write the tests before the code

TASK 1 · EASY

Your very first test

mymath.py

```
def square(n):
    return n * n
```

Your job

1. Create `test_task1.py`
2. Write **3 tests** for `square()`: a positive number, zero, and a negative number
3. Run `pytest -v` and check you see 3 PASSED

Starter: copy this and fill in the blanks

```
from mymath import square

def test_square_positive():
    assert square(4) == ___

def test_square_zero():
    ...

def test_square_negative():
    ...
```

> **Hint:** What should `square(-3)` return? Many students forget negative inputs.

TASK 2 · EASY

Find the bug

mymath.py (has a bug!)

```
def is_even(n):
    return n % 2 == 1
```

```
$ pytest test_task2.py
test_task2.py FF                                                  [100%]
...
E       assert False is True
E        +  where False = is_even(4)
...
==================== 2 failed in 0.02s ====================
```

Your job

1. Write 2 tests: `is_even(4)` should be `True`, `is_even(7)` should be `False`
2. Run pytest. Both tests should **fail**
3. Read the `E` lines in the output carefully
4. Fix the bug in `mymath.py` and run again until both pass

> **Hint:** Do not change your tests to make them pass. The tests describe the correct behaviour; the code is what is wrong.

TASK 3 · EASY

Testing string functions

text_tools.py

```
def reverse_string(text):
    return text[::-1]


def count_vowels(text):
    return sum(1 for ch in text.lower() if ch in "aeiou")
```

Your job

1. Write **2 tests** for `reverse_string()`: a normal word, and an empty string
2. Write **3 tests** for `count_vowels()`: a normal word, a word in CAPITALS, and a word with no vowels
3. Give every test a name that says what it checks

> **Hint:** `count_vowels("Pakistan")` should be 3. Try `"rhythm"` for a word with no vowels.

TASK 4 · EASY

Testing errors with pytest.raises

list_tools.py

```python
def find_max(numbers):
    if not numbers:
        raise ValueError("List is empty")
    biggest = numbers[0]
    for n in numbers:
        if n > biggest:
            biggest = n
    return biggest
```

Your job

1. Write **3 tests** that check the right maximum is returned: positive numbers, negative numbers, and a list with a single item
2. Write **1 test** that checks an empty list raises `ValueError`
3. **Bonus:** also check the error message using `match=`

Starter: copy this and fill in the blanks

```python
import pytest
from list_tools import find_max

def test_empty_list_raises():
    with pytest.raises(___):
        find_max([])
```

TASK 5 · MEDIUM

Comparing decimals with pytest.approx

mymath.py

```python
def celsius_to_fahrenheit(c):
    return c * 9 / 5 + 32


def average(numbers):
    return sum(numbers) / len(numbers)
```

Your job

1. Test that 100°C converts to 212°F
2. Test that 37.5°C converts to 99.5°F
3. Test that `average([0.1, 0.2, 0.3])` is 0.2
4. First try the last test **without** `pytest.approx`. What happens? Then fix it

> **Hint:** `0.1 + 0.2 + 0.3` is not exactly `0.6` inside the computer. `pytest.approx` allows a tiny difference.

TASK 6 · MEDIUM

Many inputs with parametrize

mymath.py

```python
def is_leap_year(year):
    return year % 4 == 0 and (
        year % 100 != 0 or year % 400 == 0
    )
```

Your job

1. Write **one** test function that uses `@pytest.mark.parametrize`
2. Include at least these years: 2024, 2023, 1900, 2000
3. Run `pytest -v` and notice that each year shows as a separate test

Starter: copy this and fill in the blanks

```python
import pytest
from mymath import is_leap_year

@pytest.mark.parametrize("year, expected", [
    (2024, True),
    (____, _____),
])
def test_is_leap_year(year, expected):
    assert ___
```

> **Hint:** 1900 is **not** a leap year, but 2000 **is**. Look at the rule for years divisible by 100 and 400.

TASK 7 · MEDIUM

Reusing setup with a fixture

cart.py

```python
class ShoppingCart:
    def __init__(self):
        self.items = {}

    def add_item(self, name, price, qty=1):
        if price < 0 or qty < 1:
            raise ValueError("Invalid price or quantity")
        self.items[name] = (price, qty)

    def total(self):
        return sum(p * q for p, q in self.items.values())

    def count(self):
        return sum(q for _, q in self.items.values())
```

Your job

1. Write a fixture called `cart` that returns a cart containing 2 pens at 50 and 1 notebook at 200
2. Using the fixture, test that `total()` is 300 and `count()` is 3
3. Test that a brand new cart has a total of 0
4. Test that adding an item with a negative price raises `ValueError`

> **Hint:** A test receives a fixture just by having a parameter with the same name: `def test_total(cart):`

TASK 8 · MEDIUM

Testing files with tmp_path

file_tools.py

```
def count_lines(path):
    with open(path) as f:
        return len(f.readlines())
```

Your job

1. Use the built-in `tmp_path` fixture to create a file with 3 lines, then check `count_lines()` returns 3
2. Write a second test for an empty file (should return 0)
3. Do **not** create files in your real project folder

Starter: copy this and fill in the blanks

```
from file_tools import count_lines

def test_count_lines(tmp_path):
    f = tmp_path / "notes.txt"
    f.write_text("line one\nline two\nline three\n")
    assert ___
```

TASK 9 · CHALLENGE

Write the tests first: FizzBuzz

This time there is **no given code**. You get only the description, like a real requirement from a client.

fizzbuzz(n) takes a positive whole number and returns a string:

- `"Fizz"` if n is divisible by 3
- `"Buzz"` if n is divisible by 5
- `"FizzBuzz"` if n is divisible by both 3 and 5
- otherwise the number itself as text, e.g. `"7"`

Your job

1. Write `test_task9.py` with a **parametrized** test covering at least 6 cases
2. Run it. It fails because `fizzbuzz.py` does not exist yet. That is expected
3. Now write `fizzbuzz.py` and run the tests again until all pass
4. Swap test files with a classmate and run their tests against your code

Hint: the order of the `if` checks matters. What does your code return for 15 if you check 3 first?